❤ CardioIA — Fase 7

Relatório Técnico · Coração Sob Controle: Previsão de Crises com IA

MVP final: deploy Web + Mobile + IoT em MicroPython · FIAP — Inteligência Artificial · Novembro de 2026
Equipe: Tayná Esteves (Deploy & DevOps) · João (Backend & IoT) · Endrew Alves (Interfaces) · Carlos Souza (Arquitetura & Documentação)

1. Resumo executivo

A Fase 7 transforma o modelo preditivo da Fase 6 e o assistente da Fase 5 em um produto digital funcional. Um **ESP32 simulado no Wokwi**, programado em **MicroPython**, lê frequência cardíaca, SpO_2 e temperatura a cada 5 segundos e envia as leituras por HTTPS a um **backend FastAPI publicado no Render**. O backend calcula a probabilidade de pico de risco com o modelo da F6, classifica a faixa (baixo, moderado ou alto), aplica os protocolos clínicos e devolve o resultado ao dispositivo, que acende os LEDs e atualiza o OLED. As interfaces **Web (React + Vite, na Vercel)** e **Mobile (React Native + Expo, APK Android)** consultam a API a cada 5 segundos e mostram o painel do paciente.

O fluxo foi validado de ponta a ponta em **09/11/2026**: leituras do Wokwi apareceram na Web e no APK, e o código de deploy foi incorporado à `main` do repositório oficial pelo Pull Request #4.

> **Aviso:** projeto acadêmico com dados simulados, sem finalidade de diagnóstico clínico. Toda resposta clínica da API inclui o campo `aviso` com essa ressalva.

2. Endereços da entrega

Recurso	Endereço
Aplicação Web (Vercel)	https://cardio-ia-fase7-delta.vercel.app
Backend / Swagger (Render)	https://cardioia-backend-djsj.onrender.com · /health · /docs
Build Android (Expo EAS)	https://expo.dev/accounts/taynaesteves-team/projects/cardioia/builds/03a7c9f0-903d-4607-918c-50ff9d60d865
APK (link temporário)	https://expo.dev/artifacts/eas/7tdOKuCQB1dAh2fsRM4aLjKU9222JSKtFk8syvfqzsg.apk
Simulação IoT (Wokwi)	https://wokwi.com/projects/479256586222505985
Repositório (GitHub)	https://github.com/JV-004/CardioIA-Fase7 · Pull Request #4 (merged)
Vídeo demonstrativo	**https://youtu.be/tNvCFHC8sa8** · fluxo fim a fim Wokwi → Render → Vercel/APK (4 min 22 s)

3. Escopo e objetivos

Objetivo do enunciado	Como foi atendido
Deploy Web com CI/CD	React + Vite na Vercel, com `vercel.json` para rotas SPA e `VITE_API_URL` apontando para o Render. Backend com auto-deploy a cada commit na `main`.
Distribuição Mobile	Expo SDK 58, build na nuvem pelo EAS (perfil `preview`), APK instalado e testado no emulador Android Studio.
Backend Python integrador	FastAPI conectando o ESP32, o modelo preditivo da F6 (RandomForest), os protocolos clínicos e o Watson Assistant da F5.
Transição para MicroPython	Firmware da F3 (`sketch.ino`, C++) convertido para `main.py` + `logica.py`, com feedback em LEDs e OLED.
Arquitetura final	Sensor → MicroPython → Backend Python → APIs de IA → UI (seção 4).

4. Diagrama de arquitetura final

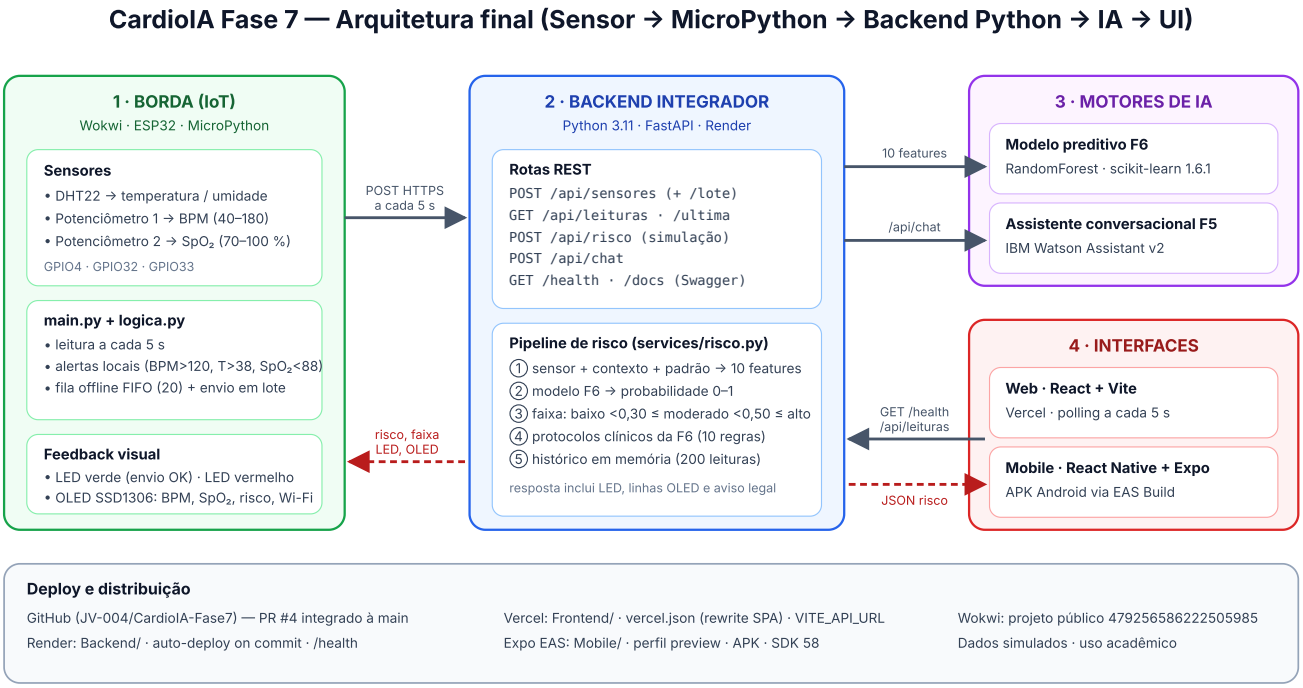


Figura 1 — Arquitetura final: borda (IoT), backend integrador, motores de IA e interfaces, com a camada de deploy.

4.1 Fluxo de dados, passo a passo

#	Etapa	O que acontece
1	Sensor	DHT22 (temperatura/umidade, GPIO4) e dois potenciômetros: BPM de 40 a 180 (GPIO32) e SpO_2 de 70 a 100 % (GPIO33).
2	MicroPython (borda)	A cada 5 s: lê os sensores, gera alertas locais (BPM > 120, T > 38 °C, SpO_2 < 88 %, umidade fora de 40–80 %), ajusta os valores aos limites da API e envia `POST /api/sensores`. Sem conexão, guarda até 20 leituras numa fila FIFO e as envia em lote (`/api/sensores/lote`) ao reconectar.
3	Backend Python	Valida o JSON (pydantic) e converte `bpm → fc_media`, `spo2 → spo2_media` e `temperatura`. As outras 7 features vêm do `contexto` ou de valores padrão de faixa normal, listados em `features_padrao`.
4	APIs de IA	O modelo da F6 devolve a probabilidade (0–1). O backend define a faixa, aplica 10 protocolos clínicos e grava a leitura no histórico. `/api/chat` repassa mensagens ao Watson Assistant v2 (F5).
5	Retorno ao hardware	A resposta traz risco, faixa, cor do LED e linhas do OLED. LED verde = envio OK; LED vermelho = alerta local ou faixa alta.
6	UI Web / Mobile	Polling a cada 5 s em `GET /health` e `GET /api/leituras?limite=20`: cards de risco e sinais vitais, protocolos acionados, dados do dispositivo e histórico.

4.2 Classificação do risco e protocolos

Faixas: BAIXO < 0,30 ≤ MODERADO < 0,50 ≤ ALTO. O corte de 0,50 é o threshold com que o modelo foi avaliado na F6. Os protocolos são regras fixas da F6, aplicadas independentemente da probabilidade: suporte respiratório (SpO_2 < 88), monitoramento intensivo (FC > 120), protocolo infeccioso (T > 38), protocolo de choque (PAS < 90), além de regras operacionais (carga do sistema, eventos em 24 h, recursos, tempo de espera, densidade de eventos). A faixa alta sempre acrescenta "Ativar estado de emergência".

5. Decisões técnicas

Decisão	Justificativa
FastAPI + pydantic	Validação automática dos dados do sensor (faixas, `NaN` → 422), documentação Swagger gerada pelo próprio código e baixa latência para o polling de 5 s.
Modelo da F6 sem alterações	Os artefatos (`predict.py`, `features.json`, `.joblib`) foram copiados como estavam. Isso exige Python 3.11 e `scikit-learn==1.6.1`, fixados no `requirements.txt` e no Render.
Features padrão explícitas	O sensor mede só 3 das 10 features. Em vez de esconder isso, a API devolve `features_padrao`, deixando claro o que foi medido e o que foi assumido.
Lógica separada do hardware	`logica.py` não depende de `machine` / `network`, então roda também em CPython e é testada com pytest (61 testes).
Edge computing com fila offline	Alertas locais funcionam sem rede. A fila FIFO de 20 leituras evita perda de dados quando o Wi-Fi cai, e o envio em lote reduz o número de conexões TLS.
Uma requisição HTTPS por ciclo	Duas requisições seguidas falhavam no Wokwi (`MBEDTLS_ERR_SSL_CONN_EOF`). Com uma só por ciclo e uma repetição em caso de falha, o envio ficou estável.
Histórico em memória	Decisão de escopo do MVP: últimas 200 leituras, sem banco de dados. Simples de manter, mas os dados se perdem ao reiniciar.
URL da API por variável	Web usa `VITE_API_URL` e Mobile usa `EXPO_PUBLIC_API_URL` (com o Render como padrão). Trocar de backend não exige mudar código, só um novo build.
Segredos fora do código	Chaves do Watson e `DEVICE_KEY` ficam no painel do Render. Uma varredura do histórico (63 commits) não encontrou credenciais.

6. Deploy e distribuição

Plataforma	Pasta	Configuração
Vercel	`Frontend/`	Build Vite (`npm run build` → `dist`); `vercel.json` reescreve todas as rotas para `/index.html`; deploy automático a partir do GitHub.
Render	`Backend/`	Web Service Python 3.11.11 · `uvicorn app.main:app --host 0.0.0.0 --port $PORT` · health check `/health` · auto-deploy on commit. O plano gratuito hiberna após ~15 min.
Expo EAS	`Mobile/`	`npx eas build --platform android --profile preview` · build `03a7c9f0…` com status *Succeeded* em 09/11/2026 (16 min 24 s) · APK testado no emulador `Medium_Phone_API_37.0`.
Wokwi	`IoT/`	Projeto público com `main.py`, `logica.py`, `ssd1306.py` e `diagram.json`; Wi-Fi `Wokwi-GUEST`; `DEVICE_ID = esp32-wokwi-01`.

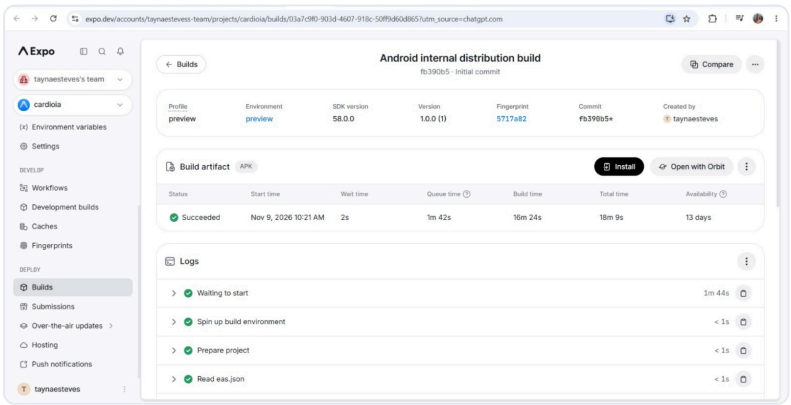


Figura 2 — Build Android no Expo EAS: perfil preview, SDK 58.0.0, artefato APK, status Succeeded.

7. Evidências de funcionamento

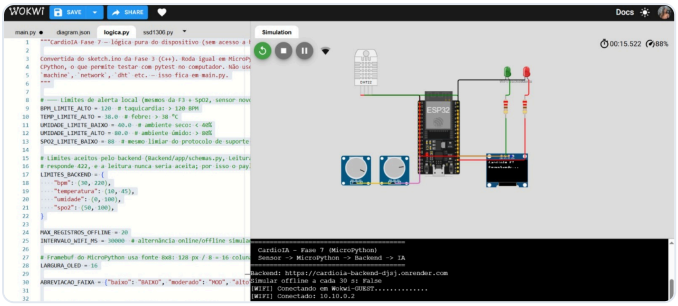


Figura 3 — Wokwi: limites de alerta em `logica.py` e console conectado ao backend no Render.

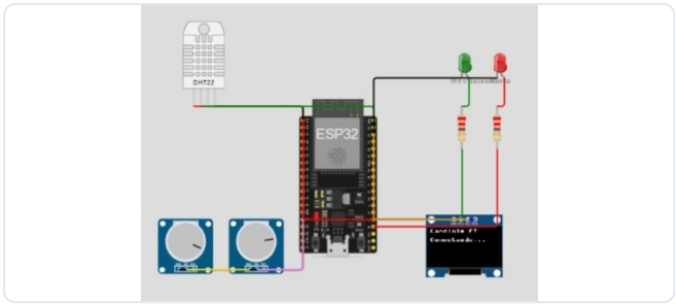


Figura 4 — Circuito: ESP32, DHT22, potenciômetros de BPM e SpO_2, LEDs e OLED SSD1306.

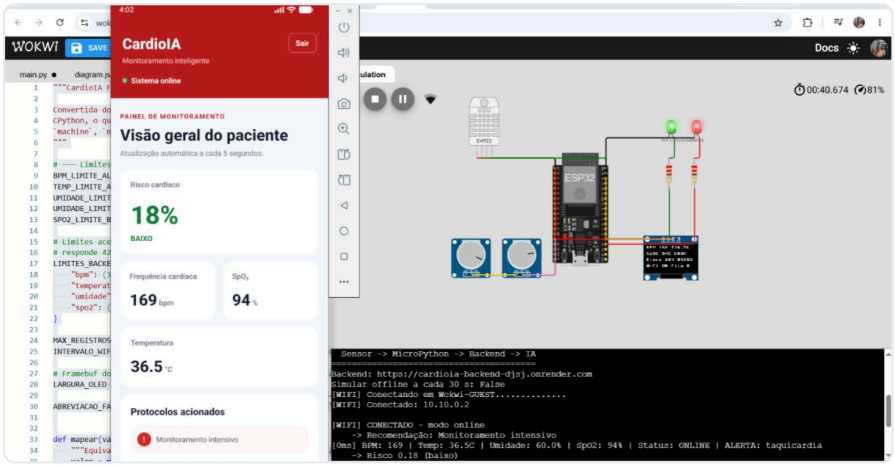


Figura 5 — Cenário A: APK no emulador ao lado do Wokwi. 169 BPM, SpO_2 94 %, 36,5 °C → risco 18 % (BAIXO), protocolo "Monitoramento intensivo"; console com `ALERTA: taquicardia` e `Risco 0.18 (baixo)`.

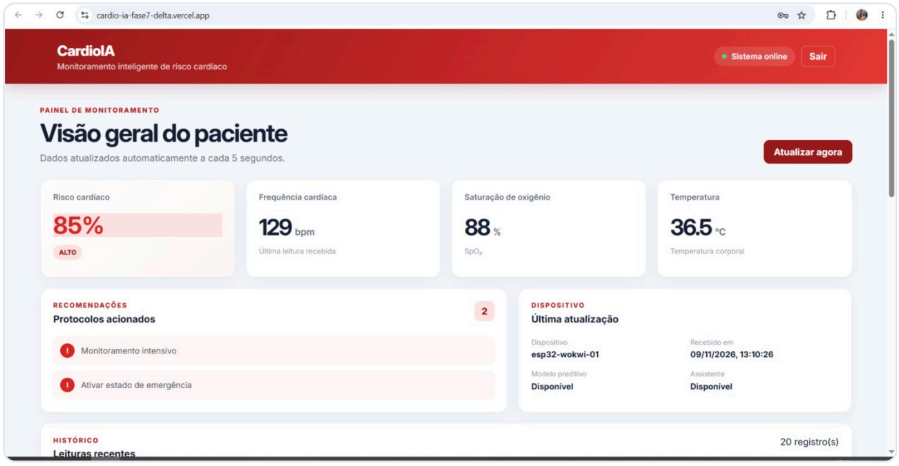


Figura 6 — Cenário B: painel Web na Vercel com leitura do `esp32-wokwi-01`. 129 BPM, SpO_2 88 %, 36,5 °C → risco 85 % (ALTO), "Monitoramento intensivo" e "Ativar estado de emergência".

Leitura dos cenários. Alertas e protocolos são regras fixas; o percentual é a probabilidade do modelo calculada sobre as 10 features. Por isso, no cenário A, o BPM acima de 120 aciona o monitoramento intensivo mesmo com risco baixo. No cenário B, a SpO_2 de 88 % fica exatamente no limite (a regra é < 88), então o suporte respiratório não é acionado, e a faixa alta acrescenta o estado de emergência.

8. Testes e validação

Verificação	Resultado
Backend — `pytest`	**90 aprovados** (reexecutado em 09/11/2026)
Lógica IoT — `pytest IoT/tests`	**61 aprovados**
Dialog skill do Watson	**Válido** — 10 nós, 23 intents, 5 entities
Publicação Web	**OK** — painel online, com leituras e protocolos (Fig. 6)
Build e execução do APK	**OK** — EAS *Succeeded*; app aberto no emulador com dados da API (Fig. 2 e 5)
Fluxo Wokwi → Render → Web/Mobile	**OK** — mudanças nos potenciômetros refletidas no painel em até ~5 s
Integração do código	**OK** — PR #4 (`feat/fase7-deploy` → `main`) incorporado

9. Problemas encontrados e soluções

Problema	Causa e solução
`MBEDTLS_ERR_SSL_CONN_EOF` no Wokwi	Conexão TLS fechada no meio da resposta. Solução: uma requisição por ciclo, repetição automática e fila offline, sem perda de dados.
Potenciômetro da F3 sempre em 4095	GND ligado a um pino inexistente (`esp:GND`). O novo `diagram.json` usa `GND.1` / `GND.2` e resistores nos LEDs.
Render lento na primeira chamada	O plano gratuito hiberna. Antes da demonstração, abrir `/health` e aguardar até ~60 s.
Cópia ZIP sem `.git`	Clone do fork na branch `feat/fase7-deploy` e Pull Request para o repositório oficial.

10. Limitações e próximos passos

- **Dados e modelo:** dados sintéticos; só 3 das 10 features são medidas pelo sensor. A acurácia clínica não foi avaliada, nem todos os cenários intermediários (faixa moderada).
- **Persistência e segurança:** histórico em memória (200 leituras) e login apenas local, de demonstração. Próximo passo: banco de dados e autenticação com token no backend.
- **Mobile:** APK validado em emulador, não em aparelho físico; o link do artefato no Expo é temporário.
- **Chat:** o backend expõe `/api/chat` (Watson F5), mas as telas Web e Mobile ainda não têm a conversa.
- **Agentes da F6:** o ponto de extensão `enriquecer_com_agentes()` está pronto em `services/risco.py` , mas o orquestrador multiagente ainda não foi ligado.

> **Itens em aberto para a submissão:** versionar `Mobile/eas.json` e incluir `android.package` no `app.json` ; deixar o repositório privado e compartilhado com o tutor; publicar o README atualizado.

11. Conclusão

A Fase 7 entrega o CardioIA como um ecossistema integrado: o dado nasce no sensor simulado, é tratado na borda em MicroPython, ganha inteligência no backend Python com o modelo da F6 e chega ao médico pela Web e pelo celular em poucos segundos. A arquitetura ficou modular — cada camada tem seu próprio deploy e pode evoluir sem afetar as demais — e documentada com contrato de API, testes automatizados e evidências de execução.